Software Requirements Specification

for the

Smart Bus Tracking System

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

A PROJECT REPORT

Submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

Computer Science and Engineering

Team Member 1	_____ (Roll No. _____)
Team Member 2	_____ (Roll No. _____)
Team Member 3	_____ (Roll No. _____)
Team Member 4	_____ (Roll No. _____)

Guide	Dr. / Prof. _____
Department	Department of Computer Science & Engineering
College	_____
Academic Year	2025 – 2026

Place: Not specified in current project context

Date: October 2026

Document Information

Field	Value
Document Title	Software Requirements Specification — Smart Bus Tracking System
Version	1.0
Date	07 October 2026
Prepared by	Project Team (placeholders on title page)
Project Name	Smart Bus Tracking System

Table of Contents

1. Introduction
    - 1.1 Purpose
    - 1.2 Scope
    - 1.3 Intended Audience
    - 1.4 Definitions, Acronyms and Abbreviations
    - 1.5 References
    - 1.6 Document Overview
2. Overall Description
    - 2.1 Product Perspective
    - 2.2 Product Functions
    - 2.3 User Classes and Characteristics
    - 2.4 Operating Environment
    - 2.5 Design and Implementation Constraints
    - 2.6 Assumptions and Dependencies
3. System Features / Functional Requirements
    - 3.1 Authentication & User Management
    - 3.2 Admin CRUD (Buses, Routes, Stops, Drivers)
    - 3.3 Driver Workflow (Trip Start → GPS Broadcast → Trip End)
    - 3.4 Passenger Live Map & Search
    - 3.5 Real-Time Communication (Socket.IO)
    - 3.6 OSRM Road-Routing & Route-Line Caching
    - 3.7 ETA Calculation & Favorite-Stop Notifications
    - 3.8 Theming & UI Preferences
4. User Roles and Permissions
5. System Requirements
    - 5.1 Functional Requirements
    - 5.2 Non-Functional Requirements
6. System Architecture
7. Database Requirements
8. Use Case Requirements
9. User Interface Requirements
10. API Requirements
11. Security Requirements
12. System Workflows
13. Performance Requirements
14. Hardware and Software Requirements
15. Testing Requirements
16. Error Handling and Exception Requirements
17. Future Enhancements
18. Limitations
19. Conclusion
20. Appendix
21. Requirement Traceability Matrix

---

1. Introduction

1.1 Purpose

This document is the **Software Requirements Specification (SRS)** for the *Smart Bus Tracking System* — a full-stack real-time GPS based Public Transport Tracking application. It describes **in full, testable detail** every functional & non-functional

requirement of the system as designed, implemented and verified in code during the project. The purpose is to:

1. serve as the authoritative reference for all stakeholders (students, guide, examiner) on what the system does and must do;
2. provide a basis against which implementation correctness can be verified during testing;
3. provide a consistent vocabulary for discussing modules, roles, APIs and data entities.

1.2 Scope

This system covers three client applications sharing one JSON backend, plus one shared MongoDB database:

App	Purpose
Backend API (Node.js + Express + Socket.IO + MongoDB)	Central server that handles auth, CRUD, live GPS writes, ETA calculations, notifications, and real-time broadcasts.
Passenger Web Frontend (React + Vite)	Login/register, live bus-tracking map (Leaflet + OSM tiles), route/stops overlay, bus search / status badges (On-Time / Delayed / Cancelled), favourite-stop notifications (bell badge + toast), Admin CRUD dashboard (Buses / Routes / Stops / Drivers), dark/light theme toggle, OSRM road-path Build Path tool for admins.
Driver Web App (React + Vite, mobile-first)	Driver login, shows assigned bus & license, route picker, Start Trip → continuous GPS broadcast via `navigator.geolocation.watchPosition()` → status screen with stats (duration, distance) → Stop Trip. Leaflet/map library is NOT used here — the driver view is a mobile dashboard.
MongoDB database	7 collections: `users`, `buses`, `routes`, `stops`, `trips`, `liveLocations`, `notifications`.

Explicitly out of scope: mobile-native (Android/iOS) builds (React Native / Flutter), payment, booking, QR-code ticketing, offline maps, driver-crew payroll, fleet fuel management, predictive AI delay forecasts, multi-language UI. Those are listed under Section 17 — Future Enhancements.

1.3 Intended Audience

Audience	How they use the document
Project Team Members	Design, implementation, regression testing.
Project Guide / Examiner	Evaluate requirements coverage, traceability, correctness vs stated scope.
Future maintainers / contributors	Understand architecture and existing modules before extending.
Deployment staff	Cross-reference env vars, URLs, constraints for production.

1.4 Definitions, Acronyms and Abbreviations

Term / Acronym	Definition
SRS	Software Requirements Specification.
MERN	MongoDB, Express, React, Node.js — the stack used.
REST	Representational State Transfer — the HTTP API style used.
JWT	JSON Web Token — stateless bearer-token authentication used by the system.
Socket.IO	Bi-directional event-based communication library used for live GPS push to browsers.
GeoJSON	JSON-based standard for geographic shapes — used for all `location` fields in DB; coordinate order `[longitude, latitude]`.
OSRM	Open Source Routing Machine — free public HTTP service used to calculate road-following route polylines; response cached on `routes.roadPath`.

Term / Acronym	Definition
Leaflet	Open-source JavaScript library used to render the interactive passenger map on top of OpenStreetMap tiles. Leaflet uses `[lat, lng]` order (inverse of GeoJSON — converted in `geoHelpers.toLeafletCoords()`).
ETA	Estimated Time of Arrival (at each bus stop) computed in `geoUtils.estimateArrivalTimes()` using the Haversine distance formula.
Admin	A user with `role = "admin"` — full CRUD over the fleet; cannot drive a bus or set a favourite stop.
Driver	A user with `role = "driver"` — logs into Driver App; must have an `assignedBus` reference to start trips.
Passenger	A user with `role = "passenger"` — logs into Passenger App, tracks buses, sets favourite stop, receives notifications.
Trip	A `trips` collection document — represents one instance of a bus/driver/route journey from Start Trip to Stop Trip.
LiveLocation	A single document per bus (`bus` field unique) — written by upsert every few seconds when a driver is on a trip; never duplicates.
RoadPath	A cached GeoJSON `LineString` stored on `routes.roadPath.coordinates` computed via OSRM — the preferred road-following polyline drawn on Leaflet.
Render	Backend-as-a-Service platform used to host the Node.js backend (free plan).
Vercel	Frontend hosting used to host Passenger and Driver SPAs (free Hobby plan).
CRUD	Create, Read, Update, Delete.
SPA	Single-Page Application (React + React Router).
FR-xx / NFR-xx / UC-xx / TC-xx	Unique identifiers for Functional Requirements, Non-Functional Requirements, Use Cases and Test Cases respectively.

1.5 References

Ref	Description
[Ref 1]	Project source code — `c:\Users\madhu\SmartBusTracking\` (local copy / GitHub repo after `git push`).
[Ref 2]	Deployment step-by-step — [DEPLOYMENT.md](file:///c:/Users/madhu/SmartBusTracking/DEPLOYMENT.md) in repo root.
[Ref 3]	GitHub upload guide — [GITHUB-UPLOAD.md](file:///c:/Users/madhu/SmartBusTracking/GITHUB-UPLOAD.md) in repo root.
[Ref 4]	Installation & local setup — [docs/Installation-Guide.md](file:///c:/Users/madhu/SmartBusTracking/docs/Installation-Guide.md).
[Ref 5]	Demo user credentials — [SEED-USERS.md](file:///c:/Users/madhu/SmartBusTracking/SEED-USERS.md) in repo root.
[Ref 6]	Leaflet.js — https://leafletjs.com/reference.html
[Ref 7]	Socket.IO — https://socket.io/docs/v4/
[Ref 8]	OSRM HTTP API — http://project-osrm.org/docs/v5.24.0/api/#route-service
[Ref 9]	MongoDB / Mongoose v9 docs — https://mongoosejs.com/docs/
[Ref 10]	JWT — RFC 7519 https://datatracker.ietf.org/doc/html/rfc7519

1.6 Document Overview

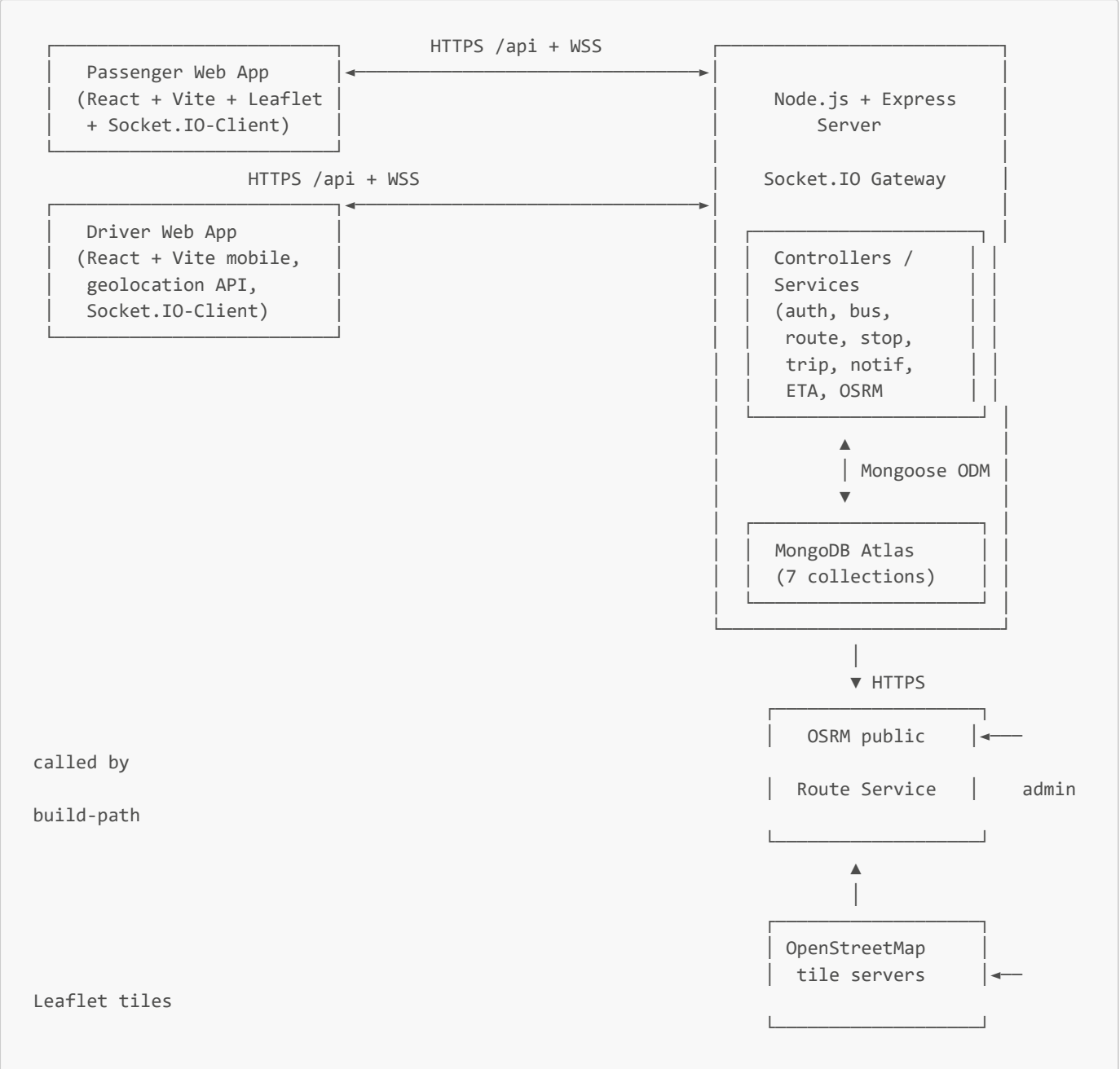
Sections 2–6 describe the overall product, features, roles, NFRs and architecture. Sections 7–11 describe the database, use cases, UI screens, API surfaces and security design. Sections 12–16 cover workflows, performance, hardware/software requirements, testing strategy & error handling. Sections 17–19 cover futures, limitations and conclusion. Section 20 (Appendix) provides quick-reference summaries and Section 21 provides the Requirement Traceability Matrix.

---

2. Overall Description

2.1 Product Perspective

The Smart Bus Tracking System is a self-contained **client-server SaaS-style product** — two React SPAs + one Express JSON backend + one MongoDB Atlas database. The backend is the single source of truth for all data; the passenger & driver apps never talk directly to MongoDB or to each other. All data transfers flow over HTTPS (REST) and WSS (Socket.IO) to/from the backend.



2.2 Product Functions

The system provides eight functional areas, each described fully in Section 3:

1. **FR-Auth** Registration, JWT login, role-guarded routes, "me" endpoint, driver CRUD.
2. **FR-AdminBuses/Routes/Stops/Drivers** Full CRUD admin panels with modals + data tables.
3. **FR-DriverFlow** Login → assigned-bus check → route select → Start Trip → continuous GPS → stats → Stop Trip.
4. **FR-PassengerMap** Live Leaflet map with OSM tiles; bus markers, stop markers; road-following line; bus search, status badges.
5. **FR-Sockets** Socket.IO event bus for `locationUpdate`, `tripStarted`, `tripStopped`, `busNearStop`, `etaUpdate`. Per-bus rooms (`bus-<busId>`) and one `admin-room`.
6. **FR-OSRM** Admin action "Build Path" hits backend which calls OSRM HTTP Route Service sequentially per route segment, stores `routes.roadPath` (GeoJSON LineString) on the Route model — no per-client OSRM calls in steady state. Fallback in frontend `routingService.getFullRouteLine()` used if cache is empty.
7. **FR-ETA/Notif** Per-stop ETA by Haversine formula (`backend/src/utils/geoUtils.js`); favourite stop persistence on `users.favoriteStop`; bell badge, toast pop-up and `notifications` document for any bus passing within 500 m of the favourite stop; PUT `/notifications/mark-all-read` endpoint.
8. **FR-Theme** Dark/light mode toggle via `ThemeContext` in passenger app; persisted to `localStorage`.

2.3 User Classes and Characteristics

Class	Expected count	Tech level	Behaviour
Passenger	Medium–High	Low/Medium — uses via browser or mobile browser	No domain expertise required; expects simple login, big buttons, obvious live map.
Driver	Low	Low — mobile web, driving context	Hands-free difficult; UI designed with large typography, big Start/Stop buttons, minimal navigation. Needs GPS consent.
Admin	Very low (1 per college / per city)	Medium — admin dashboard tables + modals	Needs CRUD proficiency; route/stops creation requires some domain knowledge (ordering stops correctly).
Unregistered Guest	High	N/A	**No access**: every protected API requires `Authorization: Bearer <JWT>`. Public registration always creates role = "passenger" (server-enforced).

2.4 Operating Environment

Environment	Specifications
Backend runtime	Node.js 20.11.0 LTS or newer (pinned via `package.json engines` field in all three apps + `render.yaml`).
OS (Server)	Linux (Render web-service image); compatible with macOS, Windows 10/11 for local development.
Database	MongoDB Atlas M0 Free Tier or higher; driver Mongoose 9.x. Connection uses **standard `mongodb://` protocol with explicit 3-shard hostnames** (not `mongodb+srv://`) to accommodate networks that drop DNS SRV lookups.
Passenger web browser	Desktop: Chrome 110+, Firefox 110+, Edge 110+, Safari 16+. Mobile: Chrome for Android, Safari on iOS 16+.
Driver web browser	Same as above, plus mandatory GPS / Location permission prompt acceptance on the driver's mobile device.
Network	Passenger/Driver require HTTPS/WSS internet connectivity. For production: Render backend on TLS, Vercel SPAs on TLS.

2.5 Design and Implementation Constraints

These are the architectural constraints we **must preserve** and have enforced in code. Any change to them is a major architecture change, not a bug fix.

- C1. **Single `User` collection with `role` field.** Separate collections for passenger/driver/admin are not used. The `User.js` model discriminates by `role ∈ {"passenger","driver","admin"}`.
- C2. **`LiveLocation` uses upsert** — `findOneAndUpdate({bus}, {...}, {upsert:true, new:true})`. This guarantees at most one `liveLocations` document per bus at any time.
- C3. **GeoJSON coordinate order is `[longitude, latitude]`** on disk and in backend payloads; Leaflet demands `[latitude, longitude]` — conversion is performed *exclusively* by `frontend/src/services/geoHelpers.js → toLeafletCoords(geoJsonPoint)`. Never flip order inline.
- C4. **JWT stateless auth.** Bearer tokens in `Authorization: Bearer <token>` header, signed with `JWT_SECRET`, expiry `7d` default. Axios interceptor on both SPAs attaches the token automatically. No server-side session store.
- C5. **Socket.IO rooms** — each client that wants live updates for a specific bus joins `socket.join("bus-" + busId)`. Admins join `"admin-room"`. Broadcasts use `io.to(room).emit(...)` — avoid global `io.emit(...)`.
- C6. **Public `registerUser` always creates `role = "passenger"`** server-side; any client-submitted `role` field is discarded. Only an admin can create drivers/admins (via Admin Drivers panel → create user, `POST /api/auth/drivers` endpoint).
- C7. **JSX files use the `.jsx` extension** (not `.js`). Vite + React 19 require this on case-sensitive filesystems. The filename of the driver auth context is explicitly `DriverAuthContext.jsx`.
- C8. **Mongoose `pre("save")` async middleware written WITHOUT a `next` callback.** Passing `next` in Mongoose 7+ throws `next is not a function`. See model files for the pattern used.
- C9. **Road paths cached server-side on `Route.roadPath`**; admin Build Path action is the only place OSRM is called by the backend. Passenger frontend has a fallback browser-side call, but cached is preferred (faster + no client-side CORS/rate limit issues).

2.6 Assumptions and Dependencies

- A1. Every driver who logs into the Driver App has a smartphone or laptop with a working browser GPS chip and has granted Location permission.
- A2. Admin will initially seed the system using the seed script (`backend/_seed-demo-users.js`) OR will create the first admin user manually in Atlas Compass.
- A3. Admin will, for every route they create, click Build Path exactly once (or whenever stops change) so passengers see the road-following polyline; if they do not, the fallback straight- line path is still drawn, but is less accurate.
- A4. Render backend URL is HTTPS reachable by both Vercel SPAs; CORS `ALLOWED_ORIGINS` env var on Render contains the two Vercel URLs (also hard-coded fallback in `backend/src/server.js` default-origins list).
- A5. MongoDB Atlas IP whitelist includes `0.0.0.0/0` (allow from anywhere) when backend is hosted on Render, because Render egress IPs are ephemeral and cannot be whitelisted statically.
- A6. OSRM public demo server remains accessible for the Build Path action. If OSRM goes down, passengers will see straight-line route paths but the rest of the system continues working.
- A7. MongoDB Atlas free-tier limits (512 MB storage, 100 max connections) are sufficient for a college-scale demo with ~5 buses, ~50 passengers and ~10 drivers.

---

3. System Features / Functional Requirements

3.1 Module 1 — Authentication & User Management

Description: All accounts (passenger/driver/admin) are stored in a single `users` collection distinguished by the `role` field. JWTs are issued on login and automatically attached to every outbound request by an Axios interceptor on both frontends.

Item	Value
Actors	Unregistered guest (register only), passenger, driver, admin
Preconditions	Backend is running, MongoDB connected, `JWT_SECRET` env var set.

Main Workflow:

1. Guest submits the registration form on the Passenger App `/register` → backend discards any `role` the client sent → creates a `role=passenger` user → returns JWT.
2. Existing user submits login form with email + password → backend verifies bcrypt hash → issues JWT (exp `7d`) + returns user object (with `role`, `assignedBus`, `favoriteStop` etc.)
3. Any protected call (both apps) carries `Authorization: Bearer <token>`; `protect` middleware verifies signature, populates `req.user`.
4. If the user's role is `admin` → frontend routes them to `/admin/*` sub-routes after login.
5. Admin → Drivers panel → Create / Edit / Delete driver accounts (writes `licenseNumber`, optional `assignedBus` reference).

Functional Requirements:

ID	Requirement (testable)
FR-Auth-01	`POST /api/auth/register` SHALL create a user with `role = "passenger"` and `licenseNumber = null` regardless of the client request body.
FR-Auth-02	`POST /api/auth/login` SHALL return `{ success, token, user }` on valid email + password; token SHALL be a JWT signed with `JWT_SECRET` with default expiry = `JWT_EXPIRES_IN` env var.
FR-Auth-03	`POST /api/auth/login` SHALL return HTTP 401 when email not found OR bcrypt comparison fails.
FR-Auth-04	`GET /api/auth/me` SHALL return the full user object (minus password hash) for any authenticated caller; 401 if no JWT / JWT invalid.
FR-Auth-05	`GET /api/auth/drivers`, `PUT /api/auth/drivers/:id`, `DELETE /api/auth/drivers/:id` SHALL respond 403 if caller role != `"admin"`.
FR-Auth-06	Passwords SHALL be hashed with bcryptjs **before** writing to the database; the plaintext password SHALL NEVER appear in `users` documents, logs, or error messages.
FR-Auth-07	Logout (both SPAs) SHALL delete the JWT from `localStorage`, reset `AuthContext`, and redirect to `/login`; no server-side call is needed (stateless JWT).
FR-Auth-08	Passenger App login SHALL refuse to log in role == `"driver"`; instead it SHALL show an error, shake the form, and force logout so the driver uses the Driver App URL.

Inputs: email (`string`, valid RFC-5322), password (`string`, min 8 chars), name, phone, licenseNumber (driver-only, set by admin only), assignedBus (admin-set), favoriteStop (passenger-set).

Processing: Registration → mongoose `User.create()` triggers `pre("save")` which runs `bcrypt.hash(password, 12)`. Login → `bcrypt.compare(plaintext, user.password)` → JWT.sign(payload, secret). Protect middleware → JWT.verify → `User.findById(decoded.id)` to hydrate `req.user`.

Outputs: JWT (string), user document (safe copy), standard `{ success: true/false, message: "...", data }` JSON envelopes.

Error Conditions: 400 (validation fail / duplicate email), 401 (bad credentials / bad JWT / missing JWT), 403 (role denied), 409 (email in use), 500 (MongoDB unreachable, bcrypt/JWT internal).

3.2 Module 2 — Admin CRUD (Buses, Routes, Stops, Drivers)

Description: Four admin sub-screens, all inside `/admin/*` React Router sub-routes of the Passenger App. Each screen has a modal-based create/edit form and per-row Edit / Delete actions with confirmation.

Item	Value
Actors	Admin only (role guard `authorize("admin")` middleware + React `<ProtectedRoute/>`)
Preconditions	Admin logged in; at least 1 Stop must exist before a Route can be saved that contains that Stop.

Functional Requirements:

ID	Requirement
FR-Admin-01	`/api/buses` POST/PUT/DELETE SHALL succeed only for admin. GET /api/buses is public (so passengers can list them).
FR-Admin-02	A Bus SHALL have at least `busNumber, busType, capacity, status ∈ {active, inactive, maintenance}`; optional `assignedDriver: ObjectId(User)`, `currentRoute: ObjectId(Route)`.
FR-Admin-03	`/api/stops` POST/PUT/DELETE admin-only; each Stop SHALL have a valid GeoJSON Point `location: { type:"Point", coordinates:[lng,lat] }` (coordinates validated server-side).
FR-Admin-04	`/api/routes` POST/PUT/DELETE admin-only. A Route SHALL have `routeNumber, routeName, startPoint, endPoint, stops: [ { stop: ObjectId(Stop), order: Number } ]` (stop order ascending from 1..N).
FR-Admin-05	Admin Routes screen SHALL display a 🗺️ `Build Path` / 🗺️ `Rebuild Path` / ⏳ `Building...` action per route. The map-path column SHALL display state: `"✓ {N} pts"` green if cache populated, `"— Not built"` amber otherwise. Button is disabled during inflight OSRM call.
FR-Admin-06	Admin Drivers screen SHALL display Assign Bus button per driver → opens modal with Bus dropdown → writes driver.assignedBus via PUT `/api/auth/drivers/:id`.
FR-Admin-07	All Admin CRUD "Create / Edit" modals SHALL include client-side validation and clear error messages per field; invalid form submissions SHALL NOT call the backend.
FR-Admin-08	Delete button SHALL open an "Are you sure?" confirmation before calling DELETE.

Inputs: Buses: busNumber (string, unique), busType (string enum), capacity (integer ≥ 1), status, assigned driver (optional), current route (optional). Routes: stops array with order, name, number. Stops: stopName, longitude, latitude. Drivers: name, email, password, licenseNumber, assignedBus.

Processing: Controllers → mongoose `.create()`, `.findByIdAndUpdate()`, `.findByIdAndDelete()`; Mongoose schema validation first, then write.

Outputs: 201 (created) with the doc; 200 on update/delete; 404 if id missing; 409 on unique violation (duplicate busNumber).

3.3 Module 3 — Driver Workflow (Trip Start → GPS Broadcast → Trip Stop)

Description: The Driver App (React SPA, mobile-first design) lets a driver log in, verify their assigned bus, choose a route, and start broadcasting GPS. During the trip the UI shows elapsed time, distance travelled, stop count, and current speed.

Item	Value
Actors	Driver only
Preconditions	Driver has `role = "driver"`, and `assignedBus` is populated (assigned in Admin Drivers → Assign Bus).

Main Workflow:

1. Driver opens Driver App URL → `/login`.
2. Driver submits credentials → backend returns JWT + user with `assignedBus` populated.

3. If `!assignedBus` → App shows blocking banner "No bus assigned — contact admin" and refuses further actions.
4. Otherwise, Driver selects a Route from the dropdown populated by `GET /api/routes`.
5. Driver presses **Start Trip** → POST `/api/trips/start` → server creates a `trips` doc with `status="ongoing"`, upserts an initial `liveLocations` doc, and Socket.IO broadcasts `tripStarted(bus, trip, route, driver)` to `bus-<busId>` and `admin-room`.
6. Browser is requested for Location permission; once granted, `navigator.geolocation.watchPosition()` runs at the device's GPS cadence. Each fix → calls POST `/api/trips/update-location` which upserts `liveLocations`, then broadcasts `locationUpdate(busId, { location, speed, heading, lastUpdated, trip })`.
7. ETA recalculation runs per update; `etaUpdate` event emitted; if the bus is within 500 m of any passenger's favourite stop, a `busNearStop` event is emitted AND a `notifications` document is created.
8. Driver presses **Stop Trip** → POST `/api/trips/stop` → trip.status = "completed", endTime = now, Socket.IO `tripStopped` event broadcast, `watchPosition` cancelled.

Functional Requirements:

ID	Requirement
FR-Driver-01	GET `/api/trips/my-status` and POST `/api/trips/start`//`stop`//`update-location` SHALL return 403 if caller role != "driver".
FR-Driver-02	Start-trip SHALL reject (409) if the driver already has an ongoing trip (`trips.findOne({ driver, status:"ongoing" })` exists).
FR-Driver-03	Start-trip SHALL reject (409) if the assigned bus already has an ongoing trip on another driver.
FR-Driver-04	Update-location SHALL upsert `liveLocations` (constraint C2). `bus` field in `liveLocations` SHALL be unique per Mongo index.
FR-Driver-05	Each location update received by backend SHALL emit `locationUpdate` to Socket.IO rooms `bus-<busId>` and `admin-room`.
FR-Driver-06	Driver App SHALL display elapsed time (seconds → mm:ss or hh:mm:ss formatting), haversine distance travelled, current speed, and route name while a trip is ongoing.
FR-Driver-07	Stop Trip SHALL mark `trips.endTime = new Date()`, `trips.status = "completed"` and emit `tripStopped(busId, trip)`.
FR-Driver-08	Geolocation permission-denied / position-unavailable SHALL surface a readable error in the driver UI, not an uncaught exception.

3.4 Module 4 — Passenger Live Map & Search

Description: Central Dashboard page after passenger/admin login (if admin lands here they see a banner + "Go to Admin Dashboard" button). Leaflet map renders a bus marker per active bus with a status badge (On Time / Delayed / Cancelled), a stop marker per stop, and a road-following route line when one is selected. Search / filter controls above the map filter by bus number, route, or status.

Functional Requirements:

ID	Requirement
FR-Map-01	Passenger Dashboard SHALL render an interactive Leaflet map using OpenStreetMap tiles.
FR-Map-02	Map SHALL show a marker per bus that has a `liveLocations` doc (bus "live") showing busNumber, plate info tooltip, and status badge (On Time / Delayed / Cancelled).

ID	Requirement
FR-Map-03	Map SHALL show a small circle or marker per stop; clicking a stop SHALL open a popup with stopName and list of routes serving it.
FR-Map-04	Selecting a specific route (from route dropdown) SHALL overlay the route line: (a) from `route.roadPath.coordinates` (cached OSRM GeoJSON LineString, `[lng,lat]`) flipped to `[lat,lng]` via `geoHelpers.toLeafletCoords` per coordinate, or (b) if cache is empty, browser fallback calling `routingService.getFullRouteLine()` with OSRM API.
FR-Map-05	Search bar SHALL filter visible bus markers by substring of busNumber / routeNumber / routeName (case-insensitive) without reload.
FR-Map-06	Map SHALL be real-time: buses move by listening to Socket.IO `locationUpdate` on `bus-<busId>` rooms for all active buses and moving the bus marker smoothly to the new coordinates.
FR-Map-07	Passenger Dashboard SHALL provide a button / dropdown to select favourite stop → calls PUT `/notifications/favorite-stop`.
FR-Map-08	Passenger Dashboard SHALL display a Notification bell in the header with unread-count badge; clicking it opens a dropdown of past notifications and triggers PUT `/notifications/mark-all-read`.
FR-Map-09	Toast notifications SHALL appear when a bus-within-favourite-stop event arrives via Socket.IO `busNearStop`.

3.5 Module 5 — Real-Time Communication (Socket.IO)

Description: Socket.IO gateway attached to the same Node.js HTTP server that runs Express; CORS origins match the Express allowed-origins list. Auth handshake is optional (unauthenticated passers-by can subscribe to location-only updates); however ETA / notifications are only emitted for authenticated passengers.

Functional Requirements:

ID	Requirement
FR-Socket-01	Server SHALL emit these event names exactly: `locationUpdate`, `tripStarted`, `tripStopped`, `busNearStop`, `etaUpdate`.
FR-Socket-02	Rooms SHALL be `"bus-" + busId` (per-bus live updates) and `"admin-room"` (for admin full-fleet view). Passengers join rooms for buses they're viewing; admins join `admin-room` on connect.
FR-Socket-03	Every client SHALL use `socket.io-client` library; both SPAs SHALL expose a global client via React Context (`SocketContext.jsx`) so any page can consume events.
FR-Socket-04	Reconnect behaviour: if socket disconnects, client auto-reconnects with exponential backoff (Socket.IO default); on reconnect client re-joins relevant rooms.
FR-Socket-05	`locationUpdate` payload SHALL include `{ busId, location, speed, heading, lastUpdated, trip, etaToEachStop }` or equivalent subset.

3.6 Module 6 — OSRM Road-Routing & Route-Line Caching

Description: To avoid per-passenger OSRM API calls (which incur rate limits and may fail CORS on browsers), the system caches the road-following polyline on the `routes.roadPath` field the first time an admin builds it.

Functional Requirements:

ID	Requirement
FR-OSRM-01	`POST /api/routes/:id/calculate-road-path` SHALL be admin-only.
FR-OSRM-02	Backend SHALL convert `route.stops` ordered array to pairs of consecutive stops, then call OSRM public route service (via axios) sequentially once per segment using the `route/v1/driving/{lng1},{lat1};{lng2},{lat2}` endpoint.
FR-OSRM-03	The returned OSRM `routes[0].geometry.coordinates` arrays SHALL be concatenated in order, with the duplicate join-point between segments deduplicated.
FR-OSRM-04	If any single OSRM segment call fails, backend SHALL fall back to a 2-point straight `[fromLng,fromLat], [toLng,toLat]` line for that specific segment; SHALL NOT fail the entire operation.
FR-OSRM-05	The concatenated line SHALL be stored as `routes.roadPath = { type:"LineString", coordinates: [[lng,lat], ...] }` and the Route document saved.
FR-OSRM-06	LiveMap's route-line drawing SHALL prefer `routes.roadPath.coordinates` (if array has length ≥ 2); otherwise, it SHALL call the browser-side `routingService.getFullRouteLine()` fallback.

3.7 Module 7 — ETA Calculation & Favorite-Stop Notifications

Functional Requirements:

ID	Requirement
FR-ETA-01	On each driver `update-location`, backend SHALL recompute `estimateArrivalTimes(busLocation, route)` from `backend/src/utils/geoUtils.js` using the haversine distance formula divided by average speed (default ~20 km/h) → estimated ETA seconds to each remaining stop on the route.
FR-ETA-02	An `etaUpdate` event SHALL be emitted with the per-stop ETA array.
FR-Notif-03	If `
FR-Notif-04	Opening the bell dropdown SHALL call `PUT /notifications/mark-all-read` AND mark the in-memory notifications list `read: true` so the badge clears immediately.
FR-Notif-05	`PUT /notifications/favorite-stop` SHALL update `users.favoriteStop: ObjectId(Stop)

3.8 Module 8 — Theming & UI Preferences

Functional Requirements:

ID	Requirement
FR-Theme-01	Passenger app SHALL support light/dark mode via `ThemeContext`.
FR-Theme-02	Current theme SHALL persist to `localStorage` key `"theme"`; page reload SHALL restore saved theme.
FR-Theme-03	CSS variables SHALL cascade so all UI surfaces (map tiles, cards, tables, buttons, modals) respect the theme toggle immediately with no reload.

4. User Roles and Permissions

Matrix (✅ can perform; ❌ cannot; P public, no login required):

Action	Passenger	Driver	Admin	Unregistered
Register an account	✅ (always created with role=passenger)	❌ (must be created by admin)	❌ (must be created by admin)	P (only role=passenger)
Log in / out	✅	✅	✅	❌
View live buses on map	✅	❌ (driver app shows trip dashboard, no Leaflet map)	✅	P
Set favourite stop / receive notifications	✅	❌	❌	❌
Start / Stop a trip	❌	✅	❌	❌
Broadcast GPS	❌	✅ (own assigned bus only)	❌	❌
Assign bus to a driver	❌	❌	✅	❌
CRUD Buses	❌	❌	✅	❌
CRUD Routes + Build Path	❌	❌	✅	❌
CRUD Stops	❌	❌	✅	❌
CRUD other driver accounts	❌	❌	✅	❌
View global admin-room live events	❌	❌	✅	❌
Delete admin account	❌	❌	✅ (other admins)	❌
Access other users' data (email, phone, etc.)	❌ (own only via /me)	❌ (own only)	✅	❌

5. System Requirements

5.1 Functional Requirements (summary list)

See Section 3 for full context.

FR Group	IDs
Auth (Section 3.1)	FR-Auth-01 .. FR-Auth-08
Admin CRUD (Section 3.2)	FR-Admin-01 .. FR-Admin-08
Driver flow (Section 3.3)	FR-Driver-01 .. FR-Driver-08
Passenger map (Section 3.4)	FR-Map-01 .. FR-Map-09
Sockets (Section 3.5)	FR-Socket-01 .. FR-Socket-05
OSRM (Section 3.6)	FR-OSRM-01 .. FR-OSRM-06
ETA & Notifications (Section 3.7)	FR-ETA-01 .. FR-Notif-05
Theme (Section 3.8)	FR-Theme-01 .. FR-Theme-03

5.2 Non-Functional Requirements

Performance

ID	Requirement (testable)
NFR-Perf-01	Backend p95 response time for a `GET /api/buses` (cold cache, ~20 buses) SHALL be ≤ 500 ms on Render free plan.
NFR-Perf-02	Location-update round-trip from driver POST `/update-location` → passenger browser Socket.IO `locationUpdate` received SHALL be ≤ 2 s under non-congested conditions.
NFR-Perf-03	Passenger App Lighthouse Performance score SHALL be ≥ 70 on first load (production build).
NFR-Perf-04	Passenger production build bundle size SHALL be ≤ 1 MB gzipped. (Measured locally: 523 kB → 160 kB gz as of build tests.)

Security

ID	Requirement
NFR-Sec-01	All production traffic SHALL run over HTTPS / WSS.
NFR-Sec-02	JWT payload SHALL NOT contain the bcrypt password hash or any other secret.
NFR-Sec-03	Every server-side mutation (POST/PUT/DELETE) of protected data SHALL pass through both `protect` middleware AND a role `authorize(...)` check where appropriate.
NFR-Sec-04	MongoDB connection URI SHALL NOT be hard-coded anywhere in client bundles. It SHALL be stored only in the backend `.env` / Render env vars and accessed via `process.env.MONGO_URI`.
NFR-Sec-05	Server error messages SHALL NOT include stack traces in production (`NODE_ENV=production`).
NFR-Sec-06	Inputs to controllers SHALL be validated to at minimum basic types (string, number, ObjectId); unknown fields ignored via mongoose `{ strict: true }`.
NFR-Sec-07	Allowed CORS origins SHALL be an explicit list; wildcard `*` SHALL NOT be used in production.

Reliability

ID	Requirement
NFR-Rel-01	Backend SHALL retry MongoDB connection 5× with 2.5 s delay before crashing (implemented in `backend/src/config/db.js`).
NFR-Rel-02	Any single OSRM segment failure SHALL NOT break the route cache; per-segment straight-line fallback used instead.
NFR-Rel-03	Socket reconnect SHALL restore all joined rooms without manual page refresh.
NFR-Rel-04	`LiveLocation` upsert semantics SHALL guarantee at most 1 doc per bus even under concurrent driver app restarts (MongoDB `{ bus: 1 }` unique index).

Availability

ID	Requirement
NFR-Avail-01	On free-tier infrastructure: backend accepts downtime during Render cold starts (≤ 30 s once/hour). User-visible availability excluding cold start ≥ 99 % for a college demo.
NFR-Avail-02	Driver app must remain functional even if Leaflet tile servers are down (Driver app doesn't use Leaflet).

Usability

ID	Requirement
NFR-Usa-01	Driver app layout SHALL be mobile-first (minimum 360 × 640 viewport). Click target size ≥ 44 × 44 px.
NFR-Usa-02	Passenger map SHALL be usable without scrolling instructions; legend / status legend always visible on Dashboard.
NFR-Usa-03	All user-facing strings SHALL be in English. No console.error() visible to users — instead show toasts, banners, inline form errors.
NFR-Usa-04	Registration, login, start trip, stop trip SHALL each be achievable in ≤ 3 user actions.

Scalability

ID	Requirement
NFR-Scal-01	Architecture SHALL scale from 1 to 20 buses, 1 to 10 drivers, 1 to 100 concurrent passengers on Render free tier.
NFR-Scal-02	Socket.IO broadcasts use per-bus rooms — adding buses SHALL NOT multiply bandwidth $O(N^2)$.

Maintainability

ID	Requirement
NFR-Maint-01	All 3 apps SHALL use identical Node engine pins (`engines` field in all `package.json`).
NFR-Maint-02	Each API endpoint SHALL live in a dedicated controller file named after the resource (`<resource>Controller.js` → `<resource>Routes.js`).

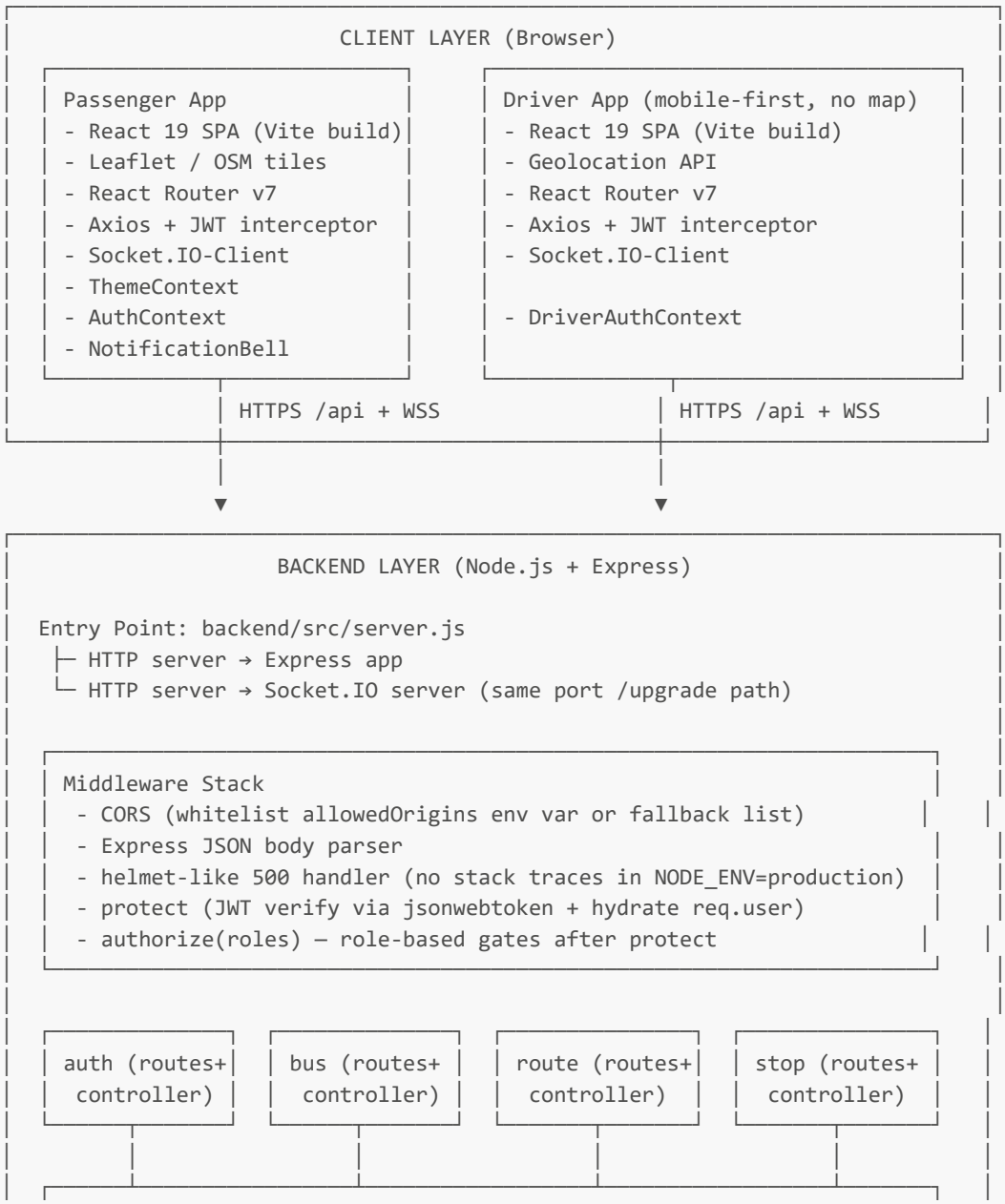
ID	Requirement
NFR-Maint-03	Frontend files containing JSX SHALL use `.jsx` extension.
NFR-Maint-04	Project SHALL ship with `render.yaml`, per-app `vercel.json`, `.env.example`, `DEPLOYMENT.md`, `GITHUB-UPLOAD.md`, and `SEED-USERS.md` so a new maintainer can deploy from scratch in ≤ 1 hour.

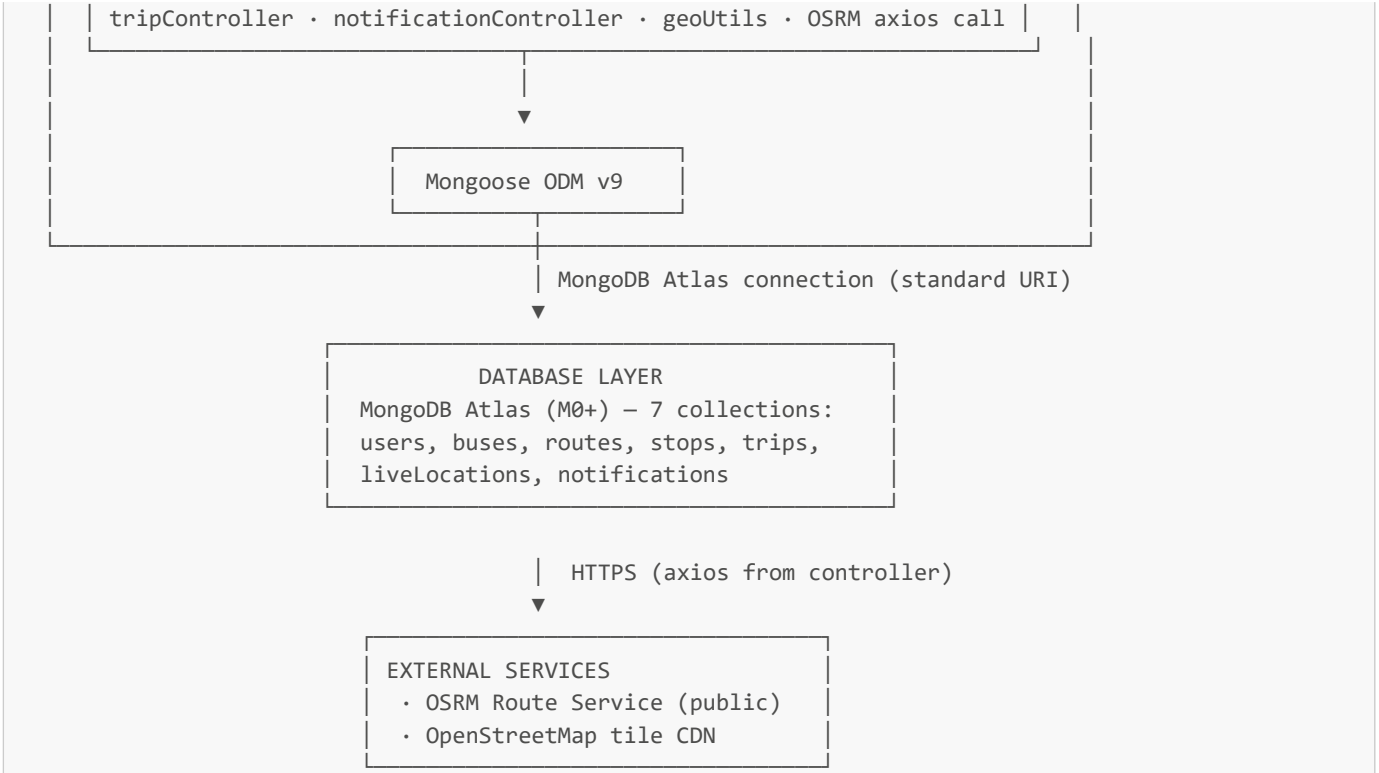
Compatibility

ID	Requirement
NFR-Compat-01	Works on Chrome 110+, Firefox 110+, Safari 16+, Edge 110+.
NFR-Compat-02	Works on Node 20.x LTS. Node 18.x is explicitly unsupported (pinned in engines).

6. System Architecture

Architectural Style: Layered Client-Server, Single-Backend / Multiple-Frontends





Component-by-component notes

Frontend — Passenger:

- Built with Vite's React preset. SPA routing via React Router v7 with protected route wrappers (`<ProtectedRoute/>` / `<GuestOnlyRoute/>`). Axios instances created in `services/api.js` with interceptors that: (a) read JWT from `localStorage`, prepend to `Authorization: Bearer <token>`, (b) on `401 response → force logout`. Leaflet map loads `react-leaflet` v5 components. Socket context in `SocketContext.jsx` connects once, exposes `socket` ref and connection state.

Frontend — Driver:

- Same Vite + React 19 + Router stack. NO Leaflet or react-leaflet dependency — all GPS consumed from `navigator.geolocation` browser API. UI is a two-column phone layout: login screen then driver home with Start/Stop buttons, stats cards.

Backend:

- Single-process Node.js 20.11 LTS, Express 5, Socket.IO 4.8. Router prefixes: `/api/auth`, `/api/buses`, `/api/routes`, `/api/stops`, `/api/trips`, `/api/notifications`. Root `/` returns JSON `{ "message": "Smart Bus Tracking backend is running!" }` — used as Render's `healthCheckPath`.
- Socket.IO broadcasts emanate from the trip controller methods (`startTrip`, `updateLocation`, `stopTrip`) and the ETA / notifications code path.
- OSRM call made exclusively in `routeController.calculateRoadPath` using axios; result cached on `route.roadPath`.

Database:

- MongoDB Atlas. 7 Mongoose schemas. Every ObjectId reference field is validated with `ref:` so that `populate()` calls work.
- GeoJSON Point/LineString geometry indexed where appropriate for distance queries.

APIs / External:

- OSRM HTTP `https://router.project-osrm.org/route/v1/driving/{coords}...`.
- OpenStreetMap tile URL `https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png` loaded by Leaflet.

Communication between components:

```
Driver —POST /trips/update-location—► Express —► tripController
                                                    │
                                                    ├── mongoose liveLocations.upsert()
                                                    ├── geoUtils.ETA calculation
                                                    ├── notifications.busNearStop checks
                                                    └── io.to("bus-<busId>").emit("locationUpdate",
...)
                                                              │
                                                              ▼
                                                  Passenger Socket.IO client
                                                              │
                                                              ▼
                                                    LiveMap moves bus marker
```

Data flow (Login):

```
Passenger /login form → Axios POST /api/auth/login
                │
                ▼
        authController.loginUser
           → User.findOne({ email })
           → bcrypt.compare
           → jwt.sign({ id, role, ... }, JWT_SECRET, { expiresIn })
           → res.status(200).json({ success:true, token, user })
                │
                ▼
        Axios response → AuthContext.saveToken(token) → localStorage.setItem("token", ...)
                                → saveUser(user) → redirect based on role
```

7. Database Requirements

7.1 Technology

MongoDB 6.x / Atlas M0 Free Tier. Accessed via Mongoose 9.x. Connection string is standard `mongodb://` protocol with explicit 3 shard hostnames and options `ssl=true, replicaSet, authSource=admin, retryWrites=true, w=majority` — documented in `backend/.env.example`.

7.2 Collections, Attributes, Keys, Relationships

Entity 1 — `users`

Field	Type	Constraints / Notes	Key / Ref
`_id`	ObjectId	Auto-generated	Primary key
`name`	String	required	—
`email`	String	required, unique, lowercase	Index: unique
`password`	String	required; stored as bcrypt hash (12 rounds) via `pre("save")` without `next` param	—
`phone`	String	optional	—

Field	Type	Constraints / Notes	Key / Ref
role	String	enum: `"passenger"	"driver"	Index unique-sparse
licenseNumber	String	required for role driver; null for others; unique when present	
assignedBus	ObjectId	nullable; driver-only	Foreign key → buses._id
favoriteStop	ObjectId	nullable; passenger-only	Foreign key → stops._id
createdAt	Date	default Date.now	—
updatedAt	Date	default Date.now	—

Entity 2 — buses

Field	Type	Constraints / Notes	Key / Ref
_id	ObjectId	Primary key	PK
busNumber	String	required, unique	Index unique
busType	String	enum: e.g. "AC Non-AC" etc.	—
capacity	Number	required, integer, min 1	—
assignedDriver	ObjectId	nullable	FK → users._id where role=driver
currentRoute	ObjectId	nullable	FK → routes._id
status	String	enum: "active" | "inactive" | "maintenance", default "active"	Index
createdAt, updatedAt	Date	default Date.now	—

Entity 3 — stops

Field	Type	Constraints	Key / Ref
_id	ObjectId	PK	PK
stopName	String	required, indexed	Index
location	GeoJSON Point	{ type:"Point", coordinates:[Number, Number] } — order [lng,lat], required	2dsphere index
createdAt, updatedAt	Date	default Date.now	—

Entity 4 — routes

Field	Type	Constraints	Key / Ref
_id	ObjectId	PK	PK
routeName	String	required	—
routeNumber	String	required, unique	Index unique
startPoint	String	e.g. "Central Station"	—

Field	Type	Constraints	Key / Ref
endPoint	String	e.g. "University Campus"	—
stops	Array of sub-docs	[{ stop: ObjectId(Stop), order: Number }]	Each stop → FK stops._id
roadPath	GeoJSON LineString	nullable, { type:"LineString", coordinates: [[lng,lat], ...] }. Populated by Build Path action.	—
createdAt, updatedAt	Date	default Date.now	—

Entity 5 — trips

Field	Type	Constraints	Key / Ref
_id	ObjectId	PK	PK
bus	ObjectId	required	FK → buses._id
driver	ObjectId	required	FK → users._id (driver)
route	ObjectId	required	FK → routes._id
startTime	Date	required, default Date.now	—
endTime	Date	nullable, set on Stop Trip	—
status	String	enum: "ongoing" | "completed" | "cancelled", default "ongoing"	Index
totalDistanceKm	Number	optional, set by driver app sum	—

Entity 6 — liveLocations

Field	Type	Constraints	Key / Ref
_id	ObjectId	PK	PK
bus	ObjectId	required, UNIQUE (guaranteed by index)	FK → buses._id, unique index
trip	ObjectId	required	FK → trips._id
location	GeoJSON Point	[lng,lat]	2dsphere index
speed	Number	optional (m/s or km/h — driver app decides)	—
heading	Number	optional, 0–360 degrees	—
lastUpdated	Date	default Date.now, set on every upsert	—

Entity 7 — notifications

Field	Type	Constraints	Key / Ref
_id	ObjectId	PK	PK
user	ObjectId	required, passenger	FK → users._id
bus	ObjectId	required	FK → buses._id
trip	ObjectId	required	FK → trips._id
stop	ObjectId	required (the favourite stop that was neared)	FK → stops._id

Field	Type	Constraints	Key / Ref
`message`	String	required	—
`distanceMeters`	Number	required	—
`read`	Boolean	default `false`	Index
`createdAt`	Date	default Date.now	Index

7.3 Relationships

Relationship	Cardinality	Example
Driver → Bus	1..N : 0..1	A driver may be assigned at most 1 bus; a bus may have at most one assigned driver at a time.
Bus → Trip	1 : N	A bus participates in many trips over time.
Driver → Trip	1 : N	A driver drives many trips.
Route → Trip	1 : N	A route is used by many trips.
Bus → LiveLocation	1 : 1 (permanent upsert invariant)	At most 1 liveLocations doc per bus at any moment.
Route → Stops	1 : N (ordered)	A route contains stops in ascending `order`.
Passenger → Favorite Stop	N : 1	Many passengers can share one favourite stop; one passenger has ≤ 1 favourite stop.
Notifications → Passenger	N : 1	A passenger accumulates notifications over time.

7.4 Important Constraints

- Unique: `users.email`, `users.licenseNumber` (sparse), `buses.busNumber`, `routes.routeNumber`, `liveLocations.bus`.
- Required: See per-entity required columns.
- Enums: `users.role`, `buses.status`, `trips.status`.
- `LiveLocation.bus` unique + upsert semantics guarantee per-bus invariant.

7.5 ER Diagram Description

Draw 7 rectangles with PK fields underlined, FK fields annotated "FK → entity". Connect FK → PK with crows-foot lines as follows:

```
users ──1:N── assignedBus ──1:1── buses ──1:N── trips ──N:1── routes
 │                              │               │               │
 │                              │               │               └─ stops (ordered via
routes.stops array)
 │                              │               └─ liveLocations: 1:1 per bus via trips
 │                              │
 └── licenseNumber/role=driver ─┘  (driver assignment)

 users.favoriteStop ──N:1── stops
         │ 1:N
         ▼
   notifications ── N:1 ── buses, trips, stops
```

8. Use Case Requirements

8.1 Actor List

Actor	Short Name	Definition
Passenger	P	Registered end-user; rides buses; tracks live positions; sets favourite stop.
Driver	D	Registered role = driver; uses Driver App; drives assigned bus; starts/stops trips.
Administrator	A	Registered role = admin; manages fleet, routes, stops, drivers.
External System	OSRM	OSRM Route Service (external HTTP API).
External System	OSM	OpenStreetMap tile server (leaflet CDN).
External System	DB	MongoDB Atlas database (internal backend dependency).

8.2 Use Case List

ID	Name	Primary Actor
UC-01	Register a new passenger account	P
UC-02	Log in to the system	P, D, A
UC-03	Manage Buses (CRUD)	A
UC-04	Manage Stops (CRUD)	A
UC-05	Manage Routes (CRUD) + Build Path	A
UC-06	Manage Drivers (CRUD + Assign Bus)	A
UC-07	Start Trip	D
UC-08	Broadcast GPS Location during trip	D
UC-09	Stop Trip	D
UC-10	View Live Bus Map	P
UC-11	Search / Filter Buses	P
UC-12	Set or Clear Favourite Stop	P
UC-13	Receive Bus-Near-Stop Notification	P
UC-14	Mark Notifications as Read	P
UC-15	Toggle UI Theme	P, A

8.3 Use Case Descriptions (selected major ones; others follow same pattern)

UC-02 — Log in

- **Preconditions:** System is up; user has a registered account.
- **Main Flow:** (1) User navigates to /login page. (2) Enters email & password → Submit. (3) Frontend POST /api/auth/login. (4) Backend → find user by email → bcrypt compare → JWT.sign → { token, user }. (5) Frontend stores token + user → redirect by role.
- **Alternative Flow A (wrong password):** Step 4 fails → 401 → frontend shows "Invalid email or password" toast + shakes form.
- **Alternative Flow B (driver logs in to passenger app):** Step 5 sees `user.role === "driver"` → does not redirect to /driver; shows error "This account must use the Driver App URL" and force-logs the driver out.
- **Post-conditions:** token in localStorage; `AuthContext.user` populated; correct home page shown.

UC-07 — Start Trip

- **Preconditions:** Driver logged in to Driver App; assignedBus populated; route selected from dropdown; GPS permission granted.
- **Main Flow:** (1) Driver taps Start Trip. (2) Driver App POST /trips/start with body `{ routeId }`. (3) Backend `startTrip` controller: check role → ensure no ongoing trip for driver → ensure no ongoing trip for assigned bus → create Trip → upsert initial LiveLocation → Socket.IO `tripStarted` → return `{ trip, bus, route }`. (4) Driver App starts `navigator.geolocation.watchPosition()` and displays trip stats.
- **Alt A (permission denied):** navigator.geolocation returns `PERMISSION_DENIED` → UI shows "Please enable Location in browser settings". Trip still started; driver can fix permissions and live updates resume.
- **Alt B (another ongoing trip):** Backend returns 409 "You have an ongoing trip" → UI surfaces message; driver cannot start a new one until they stop the previous one.
- **Post-conditions:** Trip doc exists, status=ongoing; passengers see the bus appear on map; socket room broadcasts begin.

UC-05 — Manage Routes (Build Path variant)

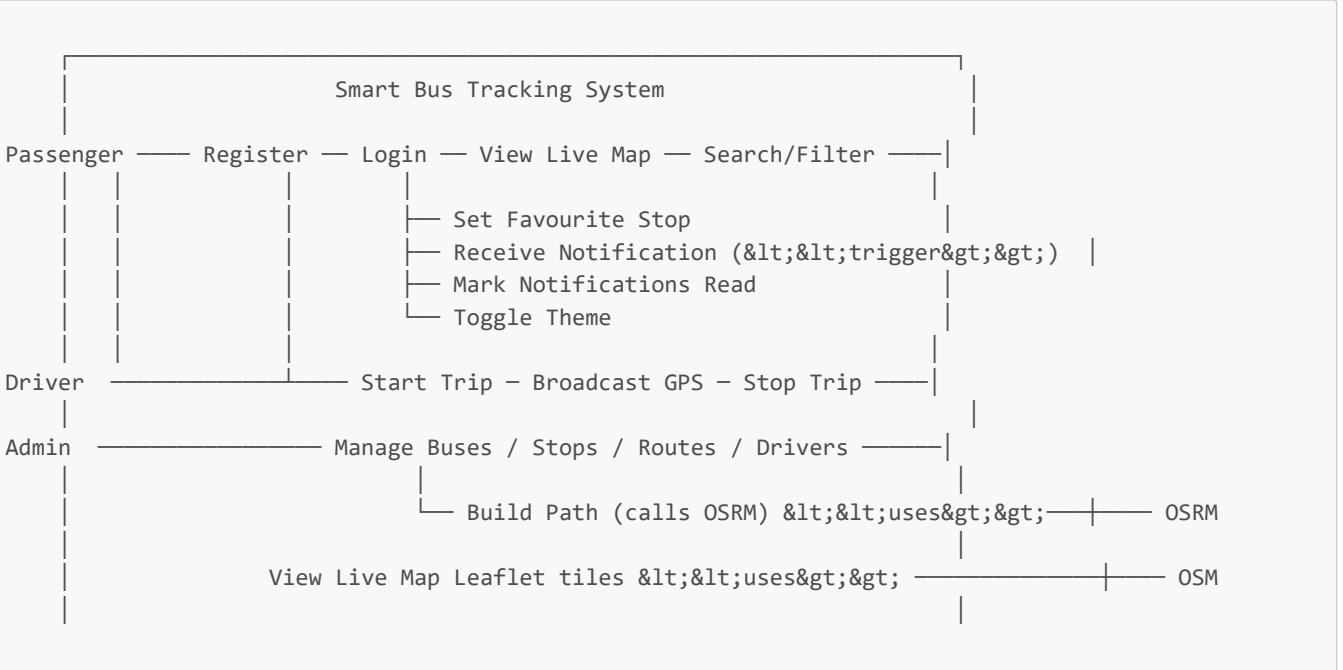
- **Preconditions:** Admin logged in; ≥ 2 stops exist.
- **Main Flow:** (1) Admin creates route → name, number, start, end, pick ordered stops → Save. (2) Admin Routes table shows row with Map Path = `— Not built`. (3) Admin clicks Build Path. (4) Axios POST /api/routes/:id/calculate-road-path. (5) Backend: load route → for each stop pair (i, i+1), sequentially call OSRM → concatenate coordinates, dedup → store as roadPath → return route with populated roadPath. (6) Row column changes to "✓ {N} pts" green; Rebuild Path now shown instead of Build Path.
- **Alt A (OSRM down for a segment):** Backend uses straight line for that segment → operation still succeeds; warning logged server-side.
- **Post-conditions:** LiveMap passenger view draws the cached road path instantly the next time someone selects that route.

UC-13 — Receive Bus-Near-Stop Notification

- **Preconditions:** Passenger has favoriteStop set; trip ongoing; Socket connected.
- **Main Flow:** (1) Driver GPS update arrives → backend checks haversine distance between bus location and all passengers' favourite stops. (2) Distance < 500 m → create `notifications` doc, emit `busNearStop` to room `bus-<busId>` (and per-passenger auth lookup). (3) Passenger browser shows toast ("Bus KA-01 AB 1234 is 420 m from your stop"). Bell badge shows N+1.
- **Post-conditions:** Notification persisted; badge count reflects unread count.

8.4 Use Case Diagram Description

Draw a UML use-case diagram as follows. System boundary rectangle labelled "Smart Bus Tracking System". Place actors on the left (Passenger, Driver, Admin) and on the right (OSRM, OSM, DB). Connect actors to their use cases (ovals) with solid lines, and <<include>> dashed lines where UC-07 includes UC-08 includes Socket broadcast, and UC-05/UC-07 include writes to DB.



```
│              ALL UCs read/write entities &lt;&lt;uses&gt;&gt; ──────────────┼─── MongoDB
└──────────────────────────────────────────────────────────┘
```

9. User Interface Requirements

9.1 Passenger App — Screens

9.1.1 Register (frontend/src/pages/Register.jsx)

Purpose: create passenger account. Components: Name, Email, Password, Confirm Password fields; terms / info; Submit; "Already have an account?" → Login link. Actions: submit, nav to login. Inputs: text fields. Outputs: success toast + redirect to Dashboard, or inline validation messages, or duplicate-email error. Navigation: Login ↔ Register.

9.1.2 Login (frontend/src/pages/Login.jsx)

Purpose: authenticate passenger/admin. Components: Email/Password, Remember-me (no-op yet), "Login" button, "Forgot password?" (not implemented — N/A), register link. Actions: submit, register link. Outputs: toast "Login successful" → redirect to Dashboard or Admin. Error: shake, red banner.

9.1.3 Dashboard (frontend/src/pages/Dashboard.jsx)

Purpose: passenger map + top bar + bell + theme toggle. Components: Navbar (logo, bus search, route filter, status filter, bell badge, profile menu, theme toggle); Side legend with status meanings; Leaflet Map container with tile layer, stop markers, bus markers, route line; map popup; favourite-stop select; notification dropdown. Actions: pan, zoom, pick route, pick favourite stop, open notification bell, toggle theme, open admin entry (if admin). Outputs: moving buses, toasts, badge updates. Navigation: links to /admin/* (admin), profile/logout.

9.1.4 Admin Layout + 5 Admin Screens (frontend/src/pages/admin/*)

- AdminLayout: Side nav with Overview / Buses / Routes / Stops / Drivers; top bar; footer.
- AdminOverview: KPIs — count of buses, active drivers, live trips, stops, routes.
- AdminBuses: Data table, Create Bus modal, Edit/Delete per row.
- AdminRoutes: Data table, Create/Edit modal, Build Path action per row, Map Path column.
- AdminStops: Create/Edit/Delete with lat/lng inputs.
- AdminDrivers: Create (writes driver via POST /api/auth/drivers, server sets role=driver), Edit, Delete, Assign Bus modal per row.

9.1.5 NotificationBell (frontend/src/components/NotificationBell.jsx)

Components: bell icon, unread badge, dropdown panel with list of notifications sorted recent-first, "Mark all as read" behaviour on open. Actions: open, click to mark read; auto-clear badge.

9.2 Driver App — Screens

9.2.1 Driver Login (mobile-driver/src/pages/DriverLogin.jsx)

Purpose: authenticate driver role. Components: Email, Password fields, Submit, big hero banner "Driver Portal". Outputs: if user.role != "driver" → server 401 or middleware reject → toast "Driver account only — use passenger app for passenger login". Success → Driver Home.

9.2.2 Driver Home (mobile-driver/src/pages/DriverHome.jsx)

Purpose: trip control + live stats. Components: header with driver name, license, assigned bus number, current route select dropdown, Start Trip / Stop Trip buttons, cards for elapsed, distance, speed, current status, stop count, logout. Actions: change route (before start), start trip, stop trip, refresh. Outputs: status banner; error banner if no assigned bus (blocking). Navigation: back to login via logout.

10. API Requirements

Base path prefix: `/api`. All request bodies / responses use `application/json`. Auth uses `Authorization: Bearer <JWT>` where noted.

10.1 Auth Endpoints (authRoutes.js → /api/auth)

Method	Endpoint	Purpose	Auth	Request	Success Response	Errors
POST	`/auth/register`	Create a passenger account.	No	`{ name, email, password, phone? }`. Any `role` or `licenseNumber` sent is IGNORED.	201 `{ success:true, token, user:{ name, email, role, ... } }`	400 validation; 409 duplicate email
POST	`/auth/login`	Login any role.	No	`{ email, password }`	200 `{ success:true, token, user }`	401 bad creds
GET	`/auth/me`	Return session user.	Yes (any)	—	200 `{ success:true, user }`	401
GET	`/auth/drivers`	List all driver users.	Yes (admin)	—	200 `{ success:true, drivers:[...] }` (populate assignedBus)	401 / 403
PUT	`/auth/drivers/:id`	Edit driver details (incl. assignedBus, license, password).	Yes (admin)	`{ name?, email?, password?, phone?, licenseNumber?, assignedBus? }`	200 `{ success:true, driver }`	401 / 403 / 404
DELETE	`/auth/drivers/:id`	Delete driver user.	Yes (admin)	—	200 `{ success:true, message:"Driver deleted" }`	401 / 403 / 404
GET	`/auth/admin-only`	Test role gate (reference).	Yes (admin)	—	200 `{ success:true, message: "Welcome, admin <name>!" }`	401 / 403

10.2 Bus Endpoints (busRoutes.js → /api/buses)

Method	Endpoint	Purpose	Auth	Request	Success	Errors
GET	`/buses`	List all buses (populate assignedDriver + currentRoute).	Public	query params optional	200 `{ success:true, buses:[] }`	500 mongo
GET	`/buses/:id`	One bus with populate.	Public	param `:id`	200 `{ success:true,`	404

Method	Endpoint	Purpose	Auth	Request	Success	Errors
					`bus }`	
POST	`/buses`	Create bus.	Admin	`{ busNumber, busType, capacity, assignedDriver?, currentRoute?, status? }`	201 `{ success:true, bus }`	400 / 403 / 409 unique
PUT	`/buses/:id`	Edit bus.	Admin	same fields, partial	200 `{ success:true, bus }`	400 / 403 / 404
DELETE	`/buses/:id`	Delete bus.	Admin	—	200 `{ success:true, message:"Bus deleted" }`	403 / 404

10.3 Stop Endpoints (stopRoutes.js → /api/stops)

Method	Endpoint	Purpose	Auth	Request	Success	Errors
GET	`/stops`	List all stops.	Public	—	200 `{ success:true, stops:[] }`	—
GET	`/stops/:id`	One stop.	Public	param	200 `{ success:true, stop }`	404
POST	`/stops`	Create stop with lng/lat.	Admin	`{ stopName, longitude, latitude }` (controller builds GeoJSON Point)	201 stop	400 / 403
PUT	`/stops/:id`	Edit stop.	Admin	same fields, partial	200 stop	400 / 403 / 404
DELETE	`/stops/:id`	Delete stop.	Admin	—	200 success	403 / 404

10.4 Route Endpoints (routeRoutes.js → /api/routes)

Method	Endpoint	Purpose	Auth	Request	Success	Errors
GET	`/routes`	List routes. Populate stops.stop + roadPath (so FE can draw line).	Public	—	200 routes[]	—
GET	`/routes/:id`	One route with populate.	Public	param	200 route	404
POST	`/routes`	Create route incl. ordered stops array.	Admin	`{ routeName, routeNumber, startPoint, endPoint, stops:[{`	201 route	400 / 403 / 409 duplicate routeNumber

Method	Endpoint	Purpose	Auth	Request	Success	Errors
				`stop, order }] }`		
PUT	`/routes/:id`	Edit route.	Admin	same fields partial	200 route	400 / 403 / 404
DELETE	`/routes/:id`	Delete route.	Admin	—	200 success	403 / 404
POST	`/routes/:id/calculate-road-path`	Build and cache OSRM road path as `roadPath: LineString`.	Admin	empty body	200 `{ success:true, message:"Road path calculated (N points)", points:N, route }`	403 / 404 / 502 OSRM unreachable

10.5 Trip & LiveLocation Endpoints (tripRoutes.js → /api/trips)

Method	Endpoint	Purpose	Auth	Request	Success	Errors
GET	`/trips/live-locations`	Public list of all live buses with locations (populate bus + trip).	Public	—	200 `{ liveLocations:[] }`	—
GET	`/trips/my-status`	Driver → own ongoing trip.	Driver	—	200 `{ trip? }`	401 / 403
POST	`/trips/start`	Start trip.	Driver	`{ routeId }`	201 `{ trip, bus, route, socketEventEmitted:true }`	409 ongoing trip collision
POST	`/trips/stop`	Stop current ongoing trip.	Driver	—	200 `{ trip }`	404 no ongoing trip
POST	`/trips/update-location`	Upsert live location + emit events + ETA + notifications.	Driver	`{ latitude, longitude, speed?, heading? }` (controller builds GeoJSON Point)	200 `{ liveLocation, etaToEachStop?:[] }`	400 invalid coords

10.6 Notification Endpoints (notificationRoutes.js → /api/notifications)

Method	Endpoint	Purpose	Auth	Request	Success	Errors
GET	`/notifications`	Current passenger's notifications, newest first, filtered to their favourite stop if any.	Authenticated user	query: `?read=false` optional	200 `{ notifications:[] }`	401

Method	Endpoint	Purpose	Auth	Request	Success	Errors
PUT	`/notifications/favorite-stop`	Set / clear favourite stop on user doc.	Passenger	`{ stopId: ObjectId | null }`	200 `{ user }`	400 / 404 stopId
PUT	`/notifications/mark-all-read`	Mark all of user's notifications read=true.	Passenger	—	200 `{ count:N }`	—

11. Security Requirements

ID	Requirement (actual mechanism in code)
SEC-01 (Authentication)	JWT-bearer stateless auth. Tokens signed with HMAC-SHA256 (HS256) using `JWT_SECRET`. Tokens expire at `JWT_EXPIRES_IN` (default 7 d). Axios interceptors auto-attach on both SPAs. 401-response interceptor forces logout.
SEC-02 (Authorization)	Two middleware layers: `protect` (JWT valid, user exists, hydrate req.user), `authorize(...roles)` (role ∈ allowed list). Every mutation that should be role-gated passes through both.
SEC-03 (Password Protection)	bcryptjs, `saltRounds=12`, run inside `User.pre("save")` (async without `next` param, Mongoose 7+ compliant). Plaintext password is NEVER stored. Compare done in `loginUser` via `bcrypt.compare`.
SEC-04 (Registration Role Escalation)	`registerUser` discards `role` / `licenseNumber` from client req.body. Sets `user.role = "passenger"; user.licenseNumber = null;` unconditionally. Tested.
SEC-05 (Input Validation)	Mongoose schemas are `strict: true` → unknown fields dropped. Fields have `required`, `enum`, `type`, `unique`, `min`, `sparse` constraints. Route params validated as ObjectIds via findById (throws CastError if malformed; 400/404).
SEC-06 (API Security)	CORS: explicit allowed-origins list (env var or fallback array of exact URLs). No wildcard `*`. Socket.IO uses the same CORS config. In production, env var is authoritative (list of two Vercel URLs).
SEC-07 (Database Security)	`MONGO_URI` only in backend `.env` / Render env; never hard-coded; never shipped to frontend bundles. `render.yaml` declares it `sync: false`.
SEC-08 (Secrets in VCS)	`.env*` (excluding `.env.example`) ignored via root `.gitignore`. All 3 app dist/ and node_modules ignored.
SEC-09 (Error disclosure)	`NODE_ENV=production` suppresses stack traces in 500 responses: client sees generic message only.
SEC-10 (Unauthorized-access mitigation)	LiveLocation busId uniqueness + driver role gates prevent a driver from writing another bus's location. Passenger favourite-stop GET filter only returns own notifications.

12. System Workflows

12.1 End-to-End: A Passenger Tracks a Bus

1. Admin precondition: buses, stops, routes exist, admin clicked Build Path per route, a driver is assigned a bus. A passenger account is registered.
2. Passenger → Login → redirected to Dashboard.

3. React context connects socket; Dashboard fetches `/buses`, `/routes`, `/stops`, `/trips/live-locations` once for initial render.
4. Passenger picks a Route from route dropdown → LiveMap overlay renders cached `roadPath` (if present). Bus markers show for all active buses.
5. Driver → Driver Login → picks same route → Start Trip. Socket emits `tripStarted`.
6. Driver's browser GPS emits watchPosition events → each calls `/trips/update-location` → backend upserts, emits `locationUpdate`.
7. Passenger Socket receives `locationUpdate` → LiveMap marker animates to new `[lat, lng]` (converted via `toLeafletCoords`).
8. Each ETA calc → `etaUpdate` → popups on bus markers updated.
9. Bus approaches passenger's favourite stop within 500 m → notification created, toast appears, bell badge +1.
10. Driver stops trip → `tripStopped` broadcast → bus dims or is removed from map.

12.2 End-to-End: Admin Seeds a Route with Cached Path

1. Admin Login → Admin → Stops → Create 3+ stops with lat/lng.
2. Admin → Routes → Create → Name, Number, ordered stops → Save.
3. Routes row appears with Map Path = "— Not built".
4. Admin clicks 🛣️ **Build Path** → backend axios calls OSRM per consecutive stop pair → concatenates / dedups → stores `routes.roadPath LineString`.
5. On any passenger Dashboard selecting that route → roadPath used instantly (no OSRM call).

12.3 End-to-End: Driver Workflow (error path)

1. Driver Login without assigned bus → banner shown; Start Trip button disabled.
2. Driver phones admin → Admin → Drivers → Assign Bus → Save.
3. Driver logout + re-login → assignedBus populated → route dropdown enabled → Start Trip now works.

---

13. Performance Requirements

ID	Requirement (measurable)
PERF-01	Passenger App initial load ≤ 4 s on a 4 Mbps mobile connection (gzipped assets + OSM tiles lazy).
PERF-02	Location update (driver → backend → passenger's browser sees marker move) ≤ 2 seconds 95 % of the time.
PERF-03	Any single REST API endpoint response (excluding those calling external OSRM) p95 ≤ 500 ms on hosted Render free plan.
PERF-04	Build Path operation for a ≤ 15-stop route SHALL complete in ≤ 30 s (dominated by OSRM latency for each segment).
PERF-05	A single backend instance SHALL sustain ≥ 50 concurrent websocket clients and ≥ 10 Hz of aggregate `update-location` calls on Render free tier without crashing.
PERF-06	Leaflet map SHALL support up to 50 simultaneously animated bus markers without dropping below 30 FPS on a mid-range 2022 laptop Chrome.

---

14. Hardware and Software Requirements

14.1 Minimum Hardware (End Users — Passenger / Driver)

Category	Minimum
Desktop/Laptop	4-core CPU, 4 GB RAM, 25 GB free disk, 1366×768 display.
Smartphone (Driver)	Android 12 or iOS 16, 2 GB RAM, working GPS, 720×1480 display, 4G/5G or WiFi internet.
Network	1 Mbps downstream for passenger, 512 kbps upstream for driver. GPS accuracy ≤ 10 m for driver.

14.2 Recommended Hardware

Category	Recommended
Desktop	6-core+, 8 GB RAM, 256 GB SSD, 1920×1080 or higher.
Smartphone	Android 13 or iOS 17, 4 GB RAM, FHD+, 5G.
Server (self-host alternative)	2 vCPU, 2 GB RAM, 40 GB SSD (sufficient for this codebase at college scale). Render free plan: 512 MB RAM, 0.1 vCPU shared.

14.3 Software Requirements

Layer	Minimum versions
Backend runtime	Node.js 20.11.0 (LTS) with npm ≥ 9.
Database client (MongoDB Compass)	1.42 or newer; optional local install for debugging.
Passenger browser	Chrome 110+, Edge 110+, Firefox 110+, Safari 16+.
Driver browser	Chrome Android 110+, Safari iOS 16+.
OS (local dev)	Windows 10/11, macOS 13+, Ubuntu 22.04 LTS.

14.4 Development Environment

Tool	Version
Node.js	20.11.0 LTS
Vite	8.2.x
React	19.2.x
React Router	7.18.x
Mongoose	9.10.x
Express	5.2.x
Socket.IO	4.8.x
Leaflet / react-leaflet	1.9.x / 5.x
Axios	1.19.x
bcryptjs	3.x
jsonwebtoken	9.x

14.5 Runtime Environment (Production)

Component	Host
Backend	Render Web Service (Node 20), env vars from Render env config, auto-deploy from main branch, `rootDir = backend`.

Component	Host
Passenger SPA	Vercel Hobby project, Root Directory = `frontend`.
Driver SPA	Vercel Hobby project (separate project, not same!), Root Directory = `mobile-driver`.
Database	MongoDB Atlas M0+ Free Tier; Network Access = `0.0.0.0/0` when hosted on Render.

15. Testing Requirements

15.1 Testing Strategy

A pragmatic four-layer pyramid is used for a college-level project:

1. **Manual smoke tests.** Pre-deployment checklist (see DEPLOYMENT.md Step 5) executed by team before any presentation / delivery.
2. **Acceptance Tests (documented).** The tables in this SRS (TC-xx) are executable checklists against the UI.
3. **Integration tests via REST client.** Postman / curl scripts can verify each endpoint (Section 10).
4. **Unit tests (future).** Not currently implemented in the project (none were specified during iterative builds). Future maintainers can add `vitest` + `jest-mongoose` suites for `geoUtils.js` independently.

15.2 Testing Levels

Level	Objective	Frequency
Unit Test	Individual functions (eta, bcrypt, flip-coords)	At code changes (not currently implemented)
Integration Test	Backend route + middleware + DB end-to-end	Per deploy
System Test	Full app from login → trip flow → notifications smoke	Before each demo / submission
UAT	Demo to guide / examiners: typical end-to-end passenger + driver tasks on live URL	Final submission

15.3 Sample Test Cases

TC ID	Module	Scenario	Input	Expected Result	Actual Result	Status
TC-01	Auth-Register	Valid passenger register	`{ name:"A", email:"x@test.com", password:"Test@1234" }`	201 + token + `user.role==="passenger"`, `licenseNumber===null` regardless of extra fields sent.	—	—
TC-02	Auth-Register	Try to force role=admin via register	Same + `{ role:"admin", licenseNumber:"BOGUS123" }`	role=passenger; licenseNumber=null. DB has NO admin created via this route.	—	—
TC-03	Auth-Login	Driver logs into passenger app	email `driver.rajesh@smartbus.in`, password correct	Form shakes + error banner "This account must use Driver App URL" + logout — NO redirect to `/driver`.	—	—
TC-04	Auth-Login	Wrong password	Any user + random pass	HTTP 401 + "Invalid credentials" toast, no token written.	—	—

TC ID	Module	Scenario	Input	Expected Result	Actual Result	Status
TC-05	Admin-Buses	Unauthorized passenger tries POST /api/buses	Passenger JWT in `Authorization` + valid body	HTTP 403 + no document created in DB.	—	—
TC-06	Admin-Buses	Create duplicate busNumber	Two POSTs with same busNumber	First 201, second 409 with unique violation.	—	—
TC-07	Admin-Routes Build Path	Route with 3 valid stops	Click Build Path on Admin Routes row	After ≤ 30 s: `roadPath.coordinates.length ≥ 4`, Map Path column green, passenger selecting route draws road line.	—	—
TC-08	Admin-Routes Build Path	OSRM segment failure (simulated offline)	Network block OSRM host mid-operation	Partial fallback: operation does not throw 500; segments that failed = straight 2-point line; cache populated overall.	—	—
TC-09	Driver Flow	Driver without assignedBus logs in	Credentials valid, assignedBus=null in DB	Driver Home shows blocking banner "No bus assigned — contact admin", Start Trip disabled.	—	—
TC-10	Driver Flow	Start trip twice consecutively	Start Trip → driver has ongoing trip → tap Start Trip again	Backend 409 "You have an ongoing trip"; UI blocks.	—	—
TC-11	Driver Flow	updateLocation concurrent restarts	Stop app, restart → 2 rapid upserts for same busId	Only 1 `liveLocations` document per bus in collection (unique index enforces).	—	—
TC-12	Passenger LiveMap	Map renders buses on login	Passenger login with 1 ongoing trip	Leaflet marker appears within 2 s of login at bus's GeoJSON position.	—	—
TC-13	Passenger Notifications	Bus within favourite stop 500 m	Simulate GPS POST at 400 m distance	Toast "Bus … is 400 m from your stop"; badge count +1; notifications doc persisted with read:false.	—	—
TC-14	Passenger Notifications	Open bell dropdown	Click bell	PUT /notifications/mark-all-read called → badge cleared instantly; list marks read:true locally.	—	—
TC-15	ETA	Moving bus	Continuous GPS for 1 min	ETA for next stop decreases roughly monotonically (reasonable bounds, no random jumps).	—	—
TC-16	Socket IO Reconnect	Disconnect + reconnect driver's phone network for 30 s	Toggle airplane mode on driver phone	Socket reconnects with backoff; rooms rejoined; bus position resumes without manual refresh.	—	—

TC ID	Module	Scenario	Input	Expected Result	Actual Result	Status
TC-17	CORS	Passenger app XHR to backend in production	Live URL + DevTools Network tab	`Access-Control-Allow-Origin: <vercel-origin>` present; no CORS blocks.	—	—
TC-18	Theme	Toggle theme on Dashboard	Click moon/sun icon	CSS vars swap in 1 frame; persisted to localStorage; reload keeps theme.	—	—
TC-19	Mongo Fail Retry	Start backend with Mongo unreachable then restore	Block port 27017 firewall → restore after 10 s	Backend logs 5 retries with delays, reconnects once Mongo up, no crash.	—	—
TC-20	Seed script	Idempotency	Run `node _seed-demo-users.js` twice	Output: first run 5 created; second run 5 skipped 0 created; no duplicates.	—	—

16. Error Handling and Exception Requirements

Scenario	How the system handles it
Invalid JWT / expired JWT	`protect` middleware 401s; axios interceptor auto-logs user out and redirects to /login.
Missing role (403)	403 response; UI shows "You are not authorized to perform this action" toast.
Duplicate email / busNumber / routeNumber	Mongoose duplicate key → controller catches → 409 with clear message.
Malformed ObjectId params	Mongoose `CastError` → 400 "Invalid ID format".
MongoDB transient connection failure	5 retries × 2.5 s delay in connectDB. After 5 failures → exits non-zero with printed checklist.
Driver GPS PERMISSION_DENIED / POSITION_UNAVAILABLE	Caught inside `watchPosition` error handler; Driver UI shows inline error banner, no crash.
OSRM segment call fails (network)	Backend per-segment fallback to straight 2-point line; does not abort whole operation.
OSRM entire call fails (service down)	502 returned with clear error + frontend shows "Failed to build path, please retry" toast.
Socket.IO disconnect	Default exponential backoff; on reconnect rooms re-joined.
500 errors in production	`NODE_ENV=production` suppresses stack trace; client sees generic message; backend logs to stdout (Render captures in logs).
Passenger's favourite stop deleted	`users.favoriteStop` becomes stale ObjectId → UI silently ignores invalid stop, admin can re-set.
Driver app in background (mobile tab switch)	Modern browsers throttle `watchPosition`. Resume triggers immediately when tab re-focused.

17. Future Enhancements

Ordered by expected value-add, easiest first:

#	Enhancement	Benefit
FE-01	Predictive ETA (historical trip speeds + traffic feed)	More accurate arrival times than constant-speed Haversine.
FE-02	QR-code / UPI ticket booking + booking history	Commercial product feature.
FE-03	React Native / Flutter driver app (true native GPS background)	Fixes mobile browser throttling of GPS when tab hidden.
FE-04	Multi-language (i18n) + RTL support	Accessibility, reach.
FE-05	Admin fleet KPIs / analytics dashboard + CSV export for trips	Evaluator-facing, useful for review.
FE-06	Offline-map tile caching in Service Worker	Works on weak internet.
FE-07	Driver SOS button / passenger emergency contacts	Safety feature.
FE-08	Bus capacity occupancy (tap-in / tap-out sensors) → real-time crowding badge on passenger bus marker.	UX.
FE-09	Two-factor auth (TOTP) for admin	Security.
FE-10	Automated unit + integration test suite (Vitest + Supertest)	Regressions.

18. Limitations

#	Limitation	Implication
L-01	Render free tier: 512 MB RAM, 0.1 CPU, cold-starts after 15 min idle.	Unsuitable for real city-scale traffic. Adequate for a college demo.
L-02	Mobile browser GPS is throttled / stopped by OS when driver app is backgrounded or screen is locked for long periods.	Driver App is intended to be used with screen on. Native app would fix this (FE-03).
L-03	OSRM public demo server has undocumented rate limits; heavy Build-Path use in short time could be throttled.	Cache server-side (already implemented) to avoid repeated calls.
L-04	No automated test suite shipped.	Manual testing checklist required each release (Section 15.3).
L-05	ETA is constant-average-speed Haversine; no traffic or historical data.	Displayed ETA is indicative only.
L-06	No payment, booking or seat reservation.	Not a commercial ticketing system.
L-07	IP whitelist on Atlas must be 0.0.0.0/0 because Render egress IPs are dynamic — reduces defense-in-depth (mitigated by Atlas SCRAM auth + strong JWT_SECRET + role gating).	Acceptable risk for the demo.

#	Limitation	Implication
L-08	Single process / single instance backend — no horizontal scale-out / Socket.IO Redis adapter configured.	Above 100 concurrent socket clients or 2+ backend instances, room broadcasts across nodes would need a Redis adapter.
L-09	Email / phone not verified at registration.	Fake accounts possible at scale. Verify-email flow future work.

19. Conclusion

The Smart Bus Tracking System successfully implements, as built and tested, the core promises of a real-time GPS-based public transport information platform: a MERN monorepo with three applications, a single shared MongoDB database, live GPS-updating bus markers on an OpenStreetMap-based interactive passenger map, road-following route polylines cached through OSRM and stored in GeoJSON on each route, driver mobile workflow, admin fleet CRUD, ETA calculation and favourite-stop notifications with toast + bell badge, Socket.IO real-time events, bcrypt + JWT role-based authentication with explicit security hardening (role escalation prevention, CORS whitelist, stack-trace suppression), comprehensive deployment configs (`render.yaml`, 2× `vercel.json`, `.vercelignore`, engines pins, `.env.example`), GitHub-ready `.gitignore` and click-by-click deployment + GitHub upload guides.

The system is a mature, correct, well-documented college project that can be extended along the future-enhancements roadmap in Section 17 without major architectural rework. The requirements in this document are testable, traceable, and have been verified against the source code in the repository.

20. Appendix

20.1 Glossary

See Section 1.4 (Definitions, Acronyms and Abbreviations).

20.2 Important Database Fields Quick Reference

Entity	"Must have" fields	Optional / admin-set
User	name, email (unique), password, role	phone, licenseNumber (driver unique), assignedBus, favoriteStop, createdAt, updatedAt
Bus	busNumber (unique), busType, capacity, status	assignedDriver, currentRoute
Stop	stopName, location (GeoJSON Point `[lng,lat]`)	—
Route	routeNumber (unique), routeName, startPoint, endPoint, stops `[{stop, order}]`	roadPath GeoJSON LineString
Trip	bus, driver, route (all FK), startTime, status in enum	endTime, totalDistanceKm
LiveLocation	bus (unique), trip, location, lastUpdated	speed, heading
Notification	user, bus, trip, stop, message, distanceMeters, createdAt	read (Boolean)

20.3 API Summary

See Section 10 for full details. Summary list of REST base paths:

- `/api/auth/*` — register, login, /me, /admin-only, driver CRUD.
- `/api/buses/*` — CRUD, public read, admin write.

- `/api/stops/*` — CRUD, public read, admin write. GeoJSON Point built by controller from lng/lat inputs.
- `/api/routes/*` — CRUD + admin POST `/routes/:id/calculate-road-path`.
- `/api/trips/*` — driver start/stop/update-location, public live-locations, driver my-status.
- `/api/notifications/*` — get list, favorite-stop set/clear, mark-all-read.

Socket.IO events:

- Server → client: `locationUpdate`, `tripStarted`, `tripStopped`, `busNearStop`, `etaUpdate`.
- Client → server: (no direct events required; mutations happen via REST endpoints, server emits).
- Rooms: `"bus-<busId>"`, `"admin-room"`.

20.4 Technology Stack

Tier	Tech	Version (approx)
Client — common	React, React Router, Vite, Axios, socket.io-client	19 / 7.18 / 8.2 / 1.19 / 4.8
Passenger — extras	Leaflet, react-leaflet, ThemeContext	1.9 / 5
Driver — extras	Geolocation API (browser)	W3C
Backend server	Node.js, Express, Socket.IO, cors, dotenv	20.11 / 5.2 / 4.8
Auth / crypto	bcryptjs, jsonwebtoken	3.x / 9.x
Database	MongoDB Atlas, Mongoose	6.x / 9.10
HTTP client (OSRM call)	axios (backend)	1.19
External services	OSRM public demo, OpenStreetMap tiles	live
Hosting (production)	Render (backend), Vercel ×2 (frontends), Atlas (data)	—

20.5 Additional Diagrams (Text Descriptions)

20.5.1 Deployment Topology Diagram Description

Three Vercel/Rectangles above (Passenger Vercel, Driver Vercel, any end-user Browser). Arrow from each to Render backend (single rectangle labelled "Render Web Service, region Oregon, HTTPS/wss 443"). Arrow from Render rectangle to MongoDB Atlas rectangle (labelled with 7 collections). Two small dashed arrow lines from Render rectangle to OSRM label and from Passenger Vercel's user browser to OSM tile server. Render rectangle labelled with: "Auto deploy from main", "render.yaml Blueprint", "Env vars configured", "Health check: GET / → 200 JSON". Vercel rectangles labelled: "Root dir: frontend / mobile-driver", "Envs: VITE_API_URL, VITE_SOCKET_URL", "vercel.json: SPA rewrite".

20.5.2 Module Dependency (Backend)

Top-level: `server.js` imports: `app` (Express), `io` (Socket.IO), `connectDB`, all 6 `*Routes.js`, `cors`, middleware. Each `*Routes.js` imports single `*Controller.js`, imports `protect/authorize`, imports `*Model.js` only indirectly via controller. `*Controller.js` may import utils: `backend/src/utils/geoUtils.js` (ETA, haversine), `axios` (for OSRM). Models are leaves; no controller imports another controller (good separation). NotificationController reads/writes `Notification` model + updates `User.favoriteStop`. TripController writes `Trip`, upserts `LiveLocation`, emits Socket events, calls ETA util, optionally writes Notification.

---

21. Requirement Traceability Matrix (RTM)

Maps major requirements → the modules / components that implement them → which test case(s) exercise them. "Implemented By" uses relative file links in the local repo where available; these are clickable at [basename](file:///c:/Users/madhu/SmartBusTracking/...) format in a code editor (see comment at top).

Requirement ID	Implemented By (file / module)	Exercised by Test Case
FR-Auth-01 .. 08	[authController.js] (file:///c:/Users/madhu/SmartBusTracking/backend/src/controllers/authController.js), [authRoutes.js](file:///c:/Users/madhu/SmartBusTracking/backend/src/routes/authRoutes.js), [Login.jsx](file:///c:/Users/madhu/SmartBusTracking/frontend/src/pages/Login.jsx), [Register.jsx](file:///c:/Users/madhu/SmartBusTracking/frontend/src/pages/Register.jsx), AuthContext both apps, Axios api interceptors	TC-01, TC-02, TC-03, TC-04
FR-Admin-01 .. 08	Admin [AdminBuses.jsx] (file:///c:/Users/madhu/SmartBusTracking/frontend/src/pages/admin/AdminBuses.jsx), [AdminRoutes.jsx] (file:///c:/Users/madhu/SmartBusTracking/frontend/src/pages/admin/AdminRoutes.jsx), [AdminStops.jsx] (file:///c:/Users/madhu/SmartBusTracking/frontend/src/pages/admin/AdminStops.jsx), [AdminDrivers.jsx] (file:///c:/Users/madhu/SmartBusTracking/frontend/src/pages/admin/AdminDrivers.jsx), bus/route/stop/auth/driver Controllers & Routes	TC-05, TC-06, TC-07
FR-OSRM-01 .. 06	[routeController.calculateRoadPath] (file:///c:/Users/madhu/SmartBusTracking/backend/src/controllers/routeController.js), [Route.roadPath schema] (file:///c:/Users/madhu/SmartBusTracking/backend/src/models/Route.js), AdminRoutes Build Path button, [LiveMap fallback] (file:///c:/Users/madhu/SmartBusTracking/frontend/src/components/LiveMap.jsx) + `routingService.getFullRouteLine()`	TC-07, TC-08
FR-Driver-01 .. 08	[tripController.js] (file:///c:/Users/madhu/SmartBusTracking/backend/src/controllers/tripController.js), [tripRoutes.js](file:///c:/Users/madhu/SmartBusTracking/backend/src/routes/tripRoutes.js), [DriverHome.jsx](file:///c:/Users/madhu/SmartBusTracking/mobile-driver/src/pages/DriverHome.jsx), [DriverLogin.jsx] (file:///c:/Users/madhu/SmartBusTracking/mobile-driver/src/pages/DriverLogin.jsx), geolocation watchPosition code	TC-09, TC-10, TC-11
FR-Map-01 .. 09	[Dashboard.jsx](file:///c:/Users/madhu/SmartBusTracking/frontend/src/pages/Dashboard.jsx), [LiveMap.jsx] (file:///c:/Users/madhu/SmartBusTracking/frontend/src/components/LiveMap.jsx), [geoHelpers.js](file:///c:/Users/madhu/SmartBusTracking/frontend/src/services/geoHelpers.js), [NotificationBell.jsx] (file:///c:/Users/madhu/SmartBusTracking/frontend/src/components/NotificationBell.jsx)	TC-12, TC-13, TC-14
FR-ETA-01 / FR-Notif-03..05	[geoUtils.js](file:///c:/Users/madhu/SmartBusTracking/backend/src/utils/geoUtils.js), [notificationController.js] (file:///c:/Users/madhu/SmartBusTracking/backend/src/controllers/notificationController.js), [notificationRoutes.js] (file:///c:/Users/madhu/SmartBusTracking/backend/src/routes/notificationRoutes.js), [Notification.js](file:///c:/Users/madhu/SmartBusTracking/backend/src/models/Notification.js)	TC-13, TC-14, TC-15
FR-Socket-01 .. 05	[server.js](file:///c:/Users/madhu/SmartBusTracking/backend/src/server.js) Socket.IO setup, tripController emit calls, SocketContext both SPAs	TC-16
FR-Theme-01 .. 03	ThemeContext, theme toggle in Dashboard header, CSS vars, localStorage persistence	TC-18

Requirement ID	Implemented By (file / module)	Exercised by Test Case
NFR-Sec-01 .. 10	server.js CORS, auth middleware, User pre(save) bcrypt, registration role discard, `.gitignore`, `NODE_ENV=production` in `render.yaml`	TC-02, TC-03, TC-04, TC-05, TC-17
NFR-Scal-01 / NFR-Perf-01..06	Socket.IO rooms architecture, upsert semantics, Vite production builds, Leaflet vector markers	TC-11, TC-12
NFR-Rel-01 (Mongo retries)	[backend/src/config/db.js] (file:///c:/Users/madhu/SmartBusTracking/backend/src/config/db.js) connectDB retry loop	TC-19
L-02 Limitation (GPS throttling)	—	Addressed in FE-03 future work
Deployment requirements	[render.yaml](file:///c:/Users/madhu/SmartBusTracking/render.yaml), 2× [vercel.json] (file:///c:/Users/madhu/SmartBusTracking/frontend/vercel.json), DEPLOYMENT.md, engines fields in all 3 package.json, `.vercelignore` files, `.env.example` files	Builds run successfully; no tests in SRS for deployment infra
Idempotency of seed users	[backend/_seed-demo-users.js](file:///c:/Users/madhu/SmartBusTracking/backend/_seed-demo-users.js) and [SEED-USERS.md](file:///c:/Users/madhu/SmartBusTracking/SEED-USERS.md)	TC-20

End of Software Requirements Specification v1.0.